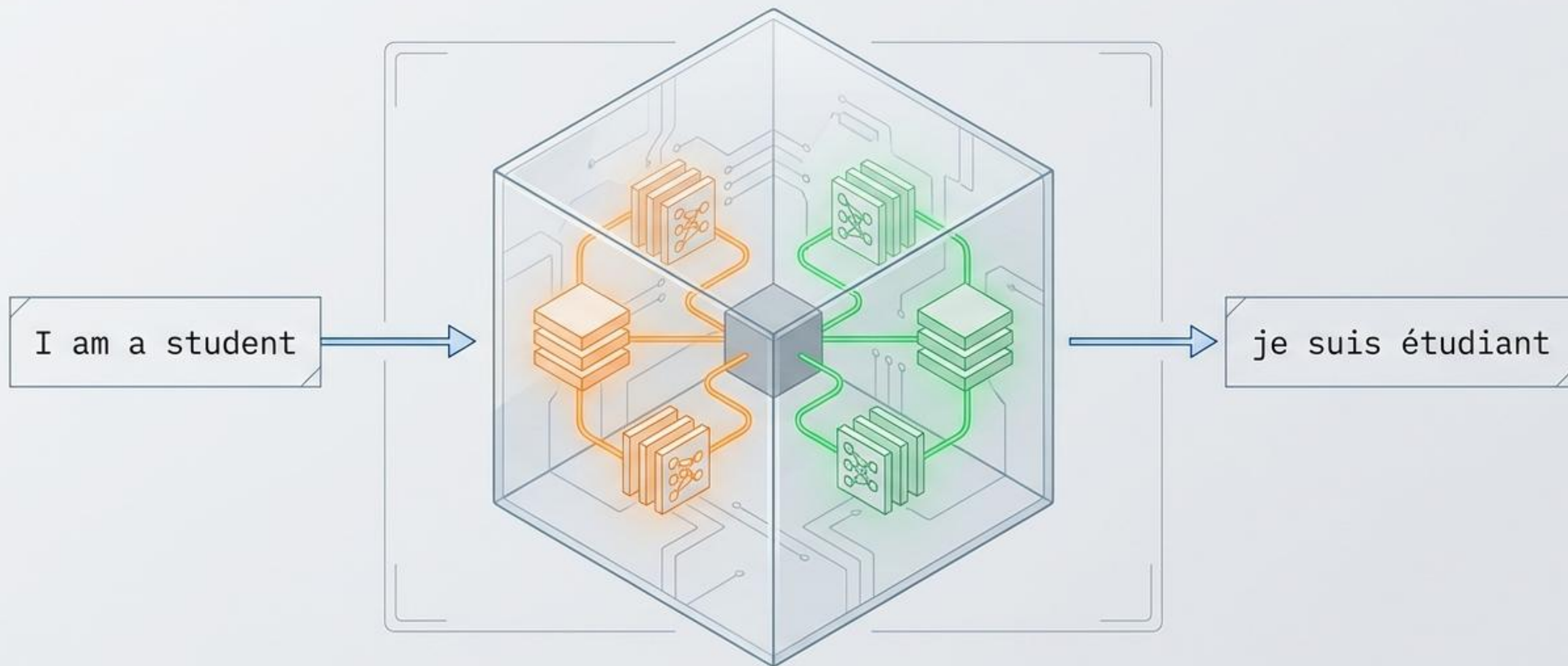


시퀀스투시퀀스(Seq2Seq)와 기계 번역의 모든 것

개념부터 파이토치(PyTorch) 구현, BLEU 스코어 평가까지의 엔드투엔드 파이프라인



블랙박스에서 글래스박스로: 기계 번역의 코어 아키텍처

블랙박스 뷰 (거시적 관점)



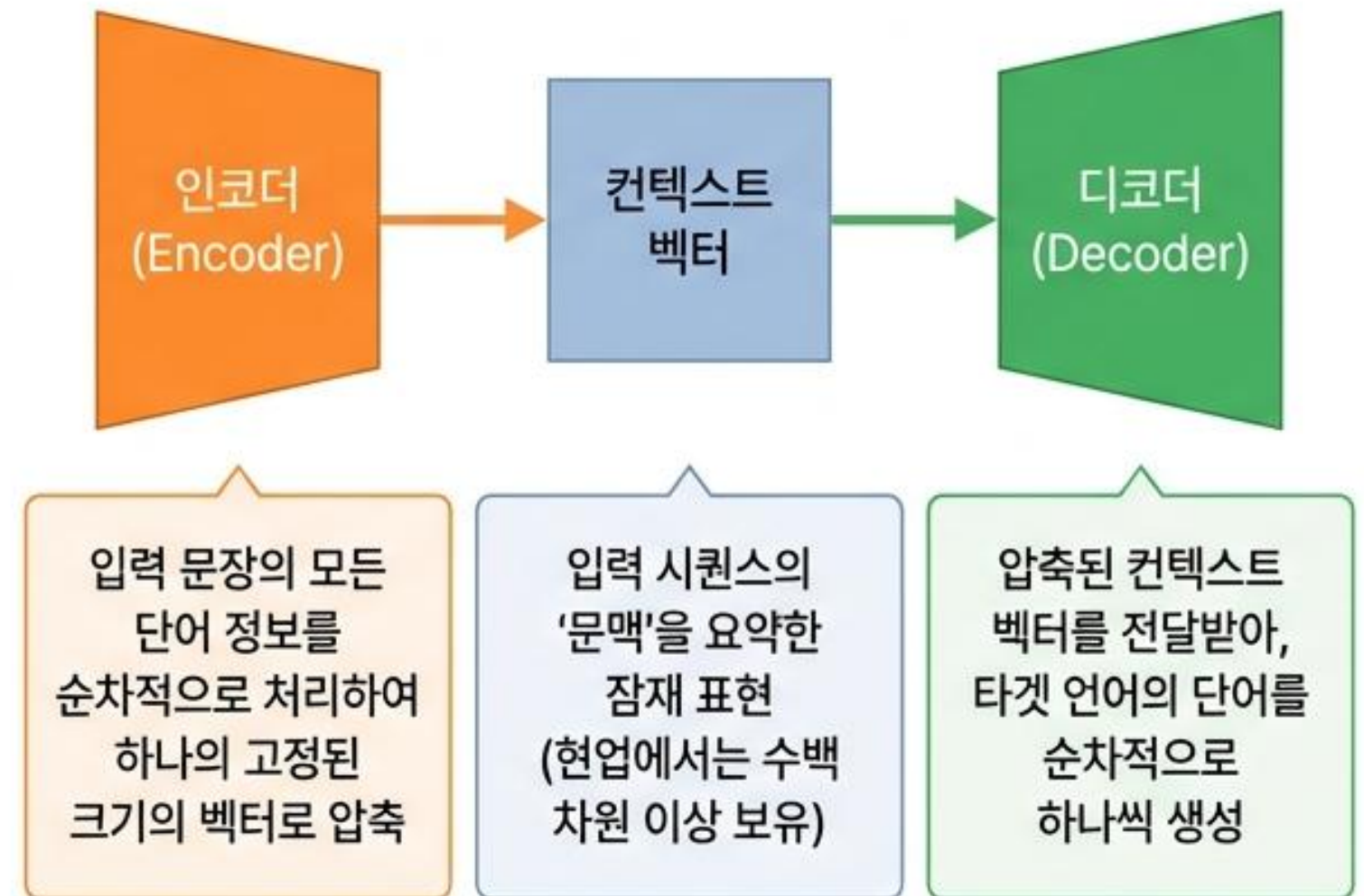
활용 도메인

챗봇
(질문-대답)

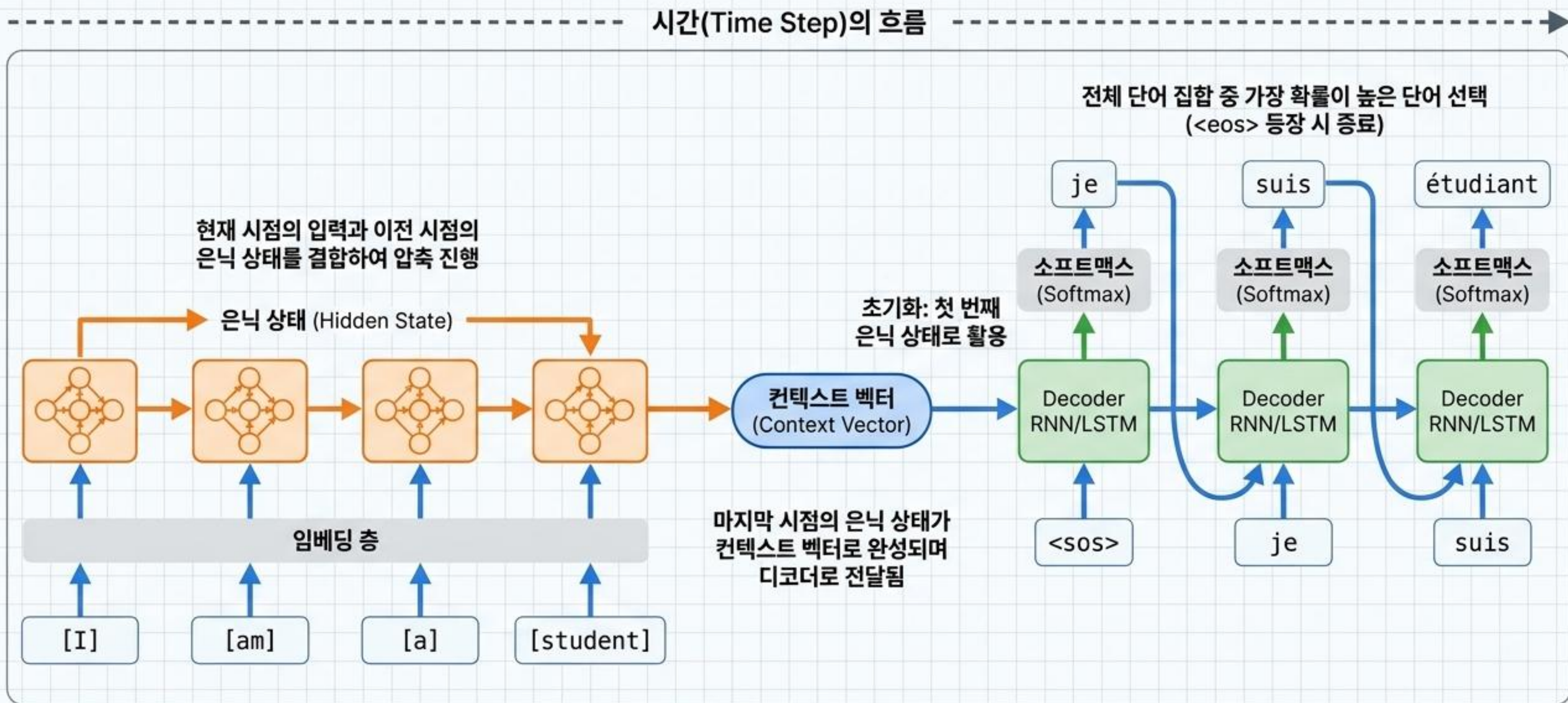
기계 번역
(원문-번역문)

텍스트 요약

글래스박스 뷰 (미시적 관점)

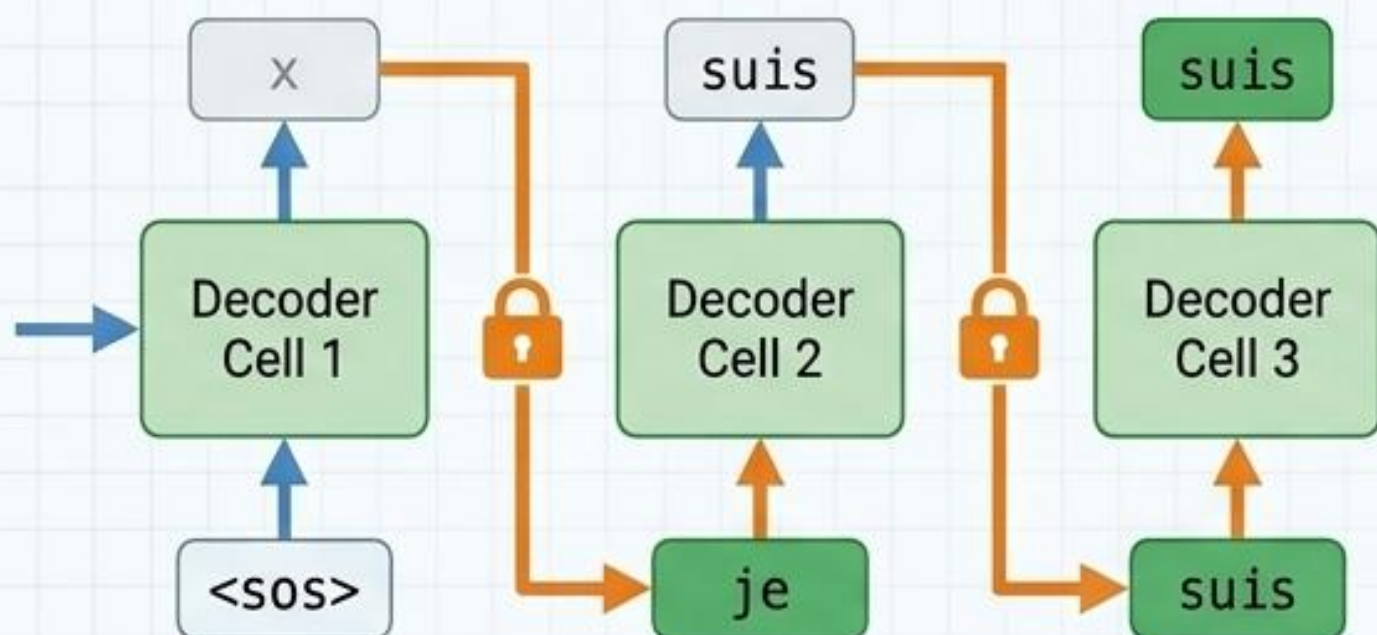


시간의 흐름을 압축하고 해제하는 RNN 셀의 연결 메커니즘



학습과 추론의 딜레마: 교사 강요(Teacher Forcing)의 필요성

훈련 단계 (Training) & 교사 강요

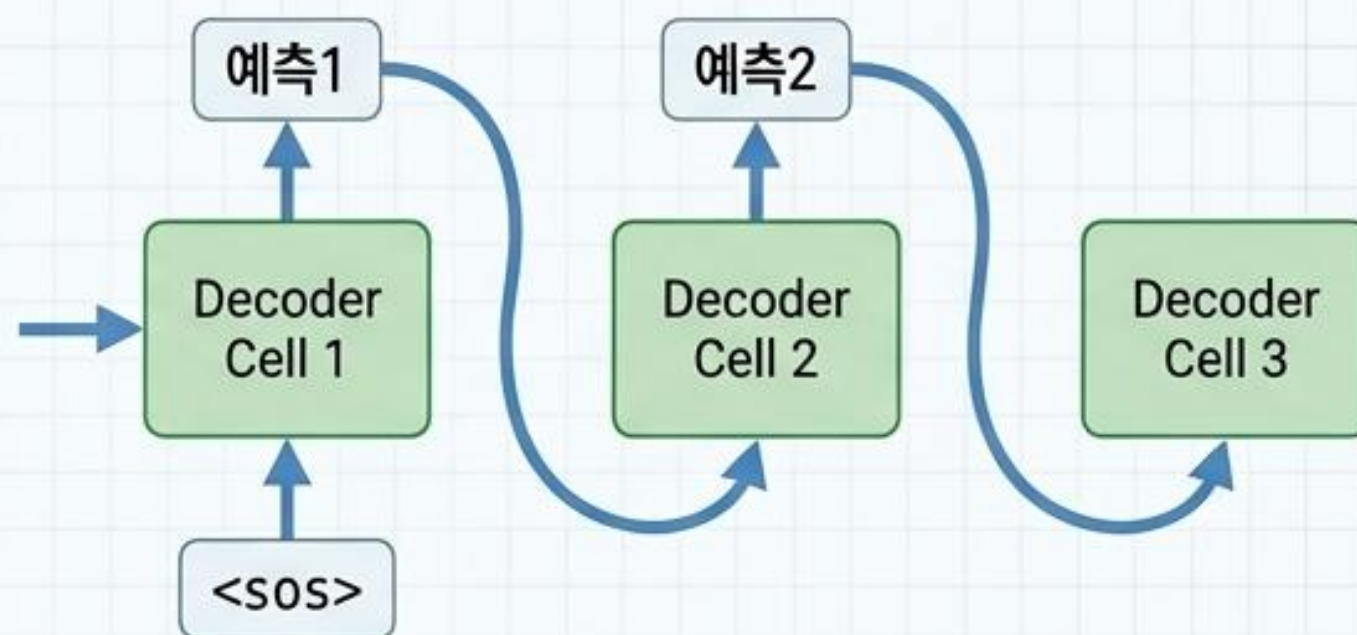


동작 방식: 디코더의 현재 시점 입력으로 이전 시점의 예측값이 아닌 실제 정답(Ground Truth)을 직접 주입

입출력 예시: 입력 <sos> je suis étudiant → 레이블
je suis étudiant <eos>

도입 이유: 이전 시점의 예측 실패로 발생하는 연쇄적인 오답(Cascading errors)을 방지하여 훈련 속도와 안정성을 대폭 향상

테스트/추론 단계 (Inference)



동작 방식: 오직 컨텍스트 벡터와 <sos>만을 입력받은 후, 스스로 예측한 단어를 다음 시점의 입력으로 재귀적 주입

입출력 예시: <sos> → 예측1 → 예측1을 주입하여 예측2 생성
→ <eos> 등장 시 반복 종료

특징: 실제 서비스 환경과 동일한 Auto-regressive 구조

[데이터 파이프라인] 노이즈 제거 및 특수 토큰 삽입

data_preprocessing.py

```
def unicode_to_ascii(s):
    # 프랑스어 악센트 삭제
    return ''.join(c for c in unicodedata.normalize('NFD', s)
                    if unicodedata.category(c) != 'Mn')

def preprocess_sentence(sent):
    sent = unicode_to_ascii(sent.lower())
    sent = re.sub(r'([?!.;])', r' \1', sent) # 구두점 분리
    sent = re.sub(r'^a-zA-Z!..?]+', r' ', sent)
    return sent

def load_preprocessed_data():
    # 텍스트 로드 및 토큰 추가
    tar_line_in = [w for w in ('<sos> ' + tar_line).split()]
    tar_line_out = [w for w in (tar_line + ' <eos>').split()]
    return encoder_input, decoder_input, decoder_target
```

전처리 전:

Avez-vous déjà diné?



preprocess_sentence()

전처리 후:

avez vous deja dine ?

Target Separation for Decoder

- 1 인코더 입력: ['go', '.']
- 2 디코더 입력: ['<sos>', 'va', '!']
- 3 디코더 레이블: ['va', '!', '<eos>']

입력에는 <sos>로 시작,
레이블에는 <eos>로 종료하여
교사 강요 세팅 준비

[데이터 파이프라인] 텍스트의 정수화 및 텐서 규격화

단어 집합(Vocabulary) 구축

```
build_vocab(sents)
# <PAD> = 0, <UNK> = 1 할당 후 빈도수 기반 정렬
```

영어 Vocab Size: 4,517 / 프랑스어 Vocab Size: 7,908



정수 인코딩 (텍스트 → 정수)

```
texts_to_sequences()
```



시퀀스 패딩 (길이 정규화)

```
pad_sequences(sentences)
# 최대 길이에 맞춰 0 (<PAD>) 채움
```



<PAD> 토큰

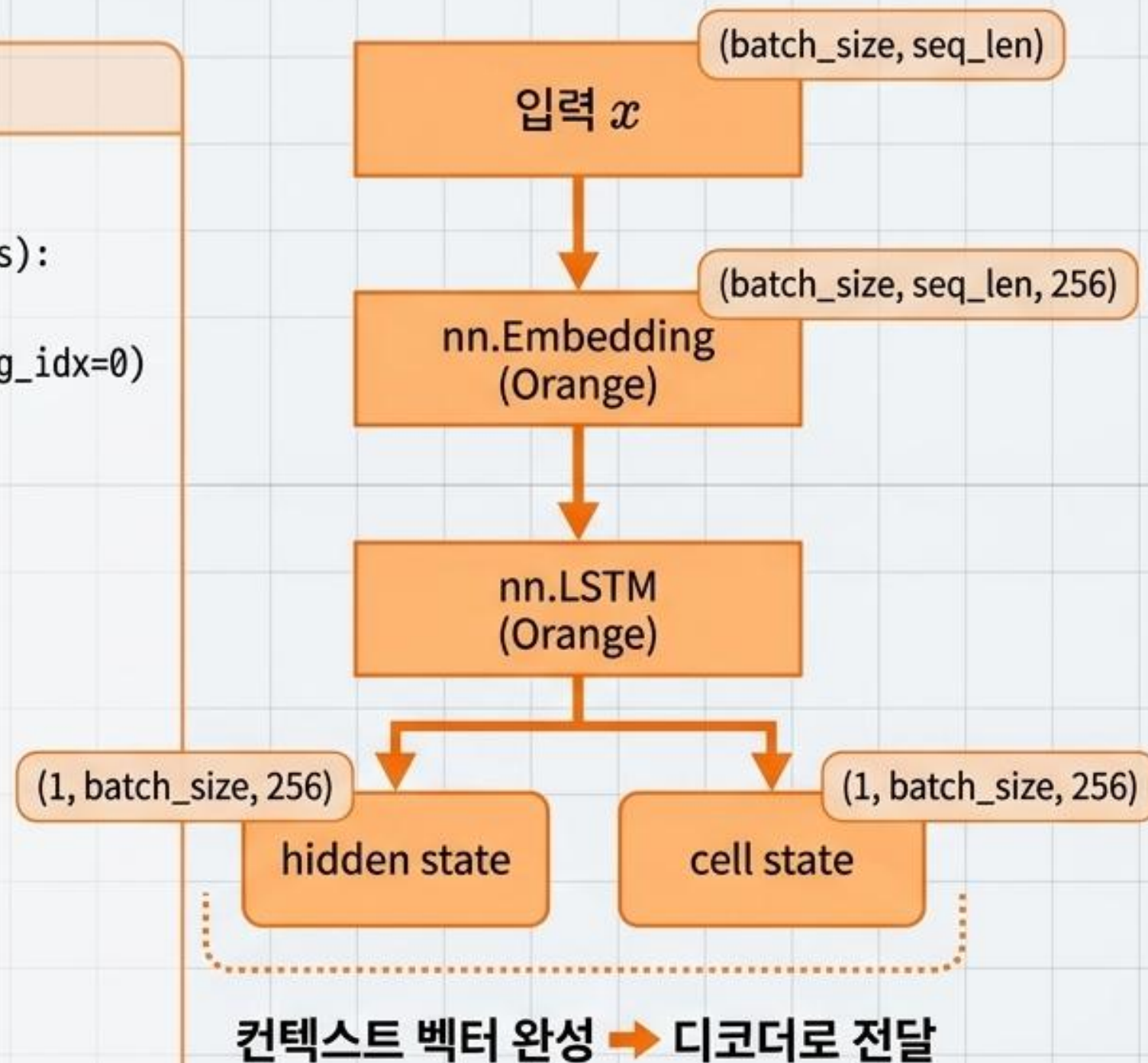
최종 텐서 형태 (Shape)

인코더 입력 (Orange)	디코더 입력 (Green)	디코더 레이블 (Green)
(33000, 7)	(33000, 16)	(33000, 16)

[파이토치 구현] 문맥을 압축하는 인코더(Encoder) 아키텍처

Encoder Class

```
class Encoder(nn.Module):  
    def __init__(self, src_vocab_size, embedding_dim, hidden_units):  
        super(Encoder, self).__init__()  
        self.embedding = nn.Embedding(src_vocab_size, 256, padding_idx=0)  
        self.lstm = nn.LSTM(256, 256, batch_first=True)  
  
    def forward(self, x):  
        x = self.embedding(x)  
        _, (hidden, cell) = self.lstm(x)  
        # 시퀀스별 출력은 버리고, 최종 요약본만 반환  
        return hidden, cell
```

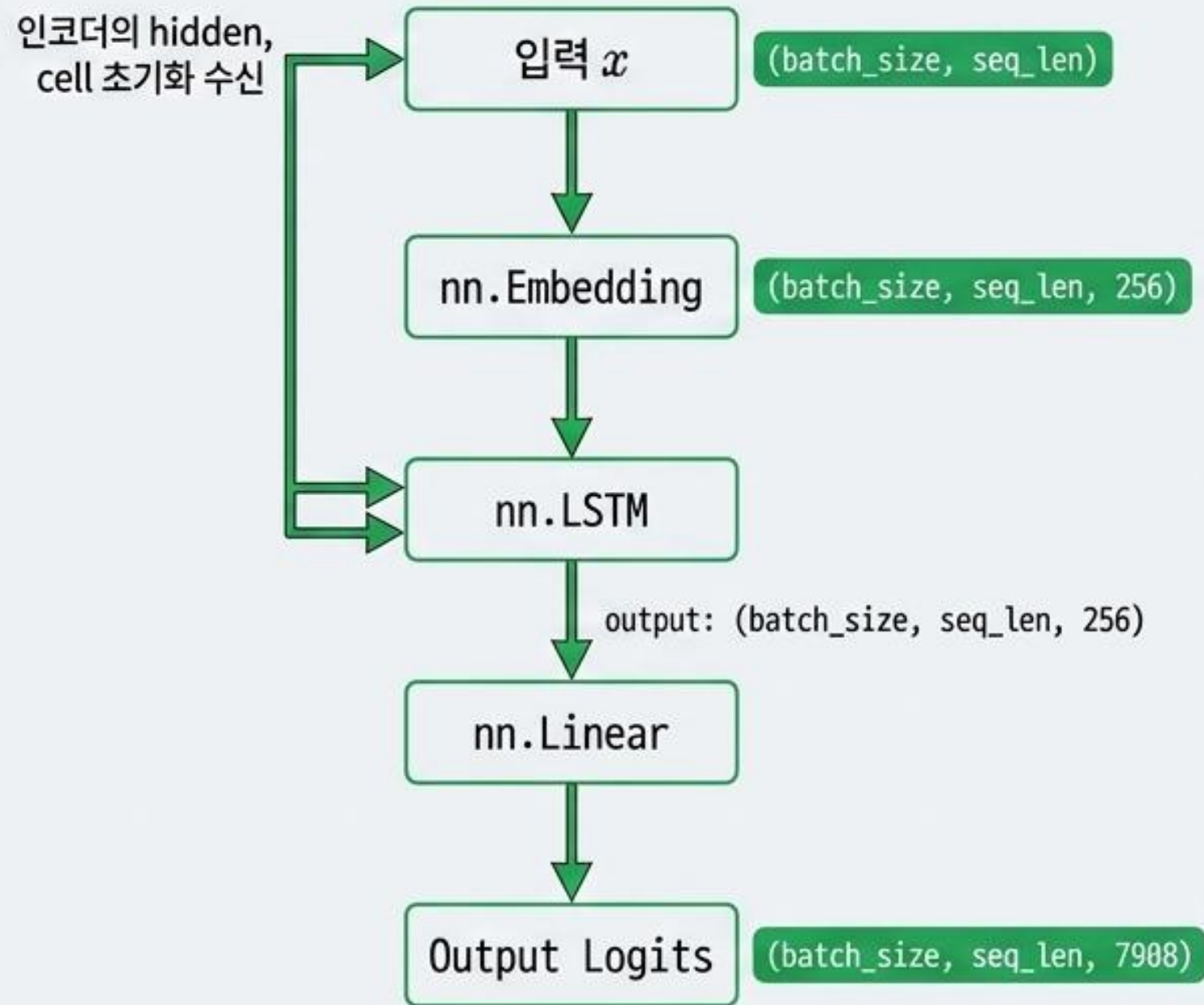


[파이토치 구현] 문맥을 해제하는 디코더(Decoder) 아키텍처

Decoder Class

```
class Decoder(nn.Module):
    def __init__(self, tar_vocab_size, embedding_dim, hidden_units):
        super(Decoder, self).__init__()
        self.embedding = nn.Embedding(tar_vocab_size, 256, padding_idx=0)
        self.lstm = nn.LSTM(256, 256, batch_first=True)
        self.fc = nn.Linear(256, tar_vocab_size)

    def forward(self, x, hidden, cell):
        x = self.embedding(x)
        output, (hidden, cell) = self.lstm(x, (hidden, cell))
        output = self.fc(output)
        return output, hidden, cell
```



매 시점마다 7,908개(프랑스어 단어 집합 크기)의
선택지 중 하나를 고르는 다중 클래스 분류 수행

[파이토치 구현] Seq2Seq 통합 라우팅 및 손실 함수 설계

Seq2Seq 통합 클래스

```
class Seq2Seq(nn.Module):  
    def forward(self, src, trg):  
        hidden, cell = self.encoder(src)  
        output, _, _ = self.decoder(trg, hidden, cell)  
        return output
```

인코더를 통한 컨텍스트 추출

디코더에 정답 데이터 `trg`와
컨텍스트 주입 (교사 강요)

손실 함수 (Loss Function)

```
nn.CrossEntropyLoss(ignore_index=0)
```

다중 클래스 분류 크로스엔트로피 사용. ignore_index=0을
설정하여 <PAD> 토큰에 대한 불필요한 오차 계산 무시.

옵티마이저 (Optimizer)

```
optim.Adam(model.parameters())
```

모델 파라미터 최적화를 위한 Adam 옵티마이저 적용.

[훈련 파이프라인] 역전파 루프와 모델 검증(Validation)

핵심 훈련 루프 및 평가

```
# 훈련 루프 핵심 로직
model.train() # 학습 모드
optimizer.zero_grad() # 기울기 초기화
outputs = model(encoder_inputs, decoder_inputs)

# 손실 계산 시 차원 평탄화
loss = loss_function(outputs.view(-1,
    outputs.size(-1)), decoder_targets.view(-1))

loss.backward() # 역전파
optimizer.step() # 가중치 업데이트

# 평가 마스킹 로직 (evaluation 함수)
model.eval() # 평가 모드 (torch.no_grad() 동반)
mask = decoder_targets != 0 # 패딩 제외
acc = ((outputs.argmax(dim=-1) ==
    decoder_targets) * mask).sum()
```

학습 결과 모니터링



```
Epoch: 1/30 | Valid Loss: 3.8343 | Valid Acc: 0.5257
...
Epoch: 13/30 | Valid Loss: 1.5129 | Valid Acc: 0.7180
Validation loss improved to 1.5129. 체크포인트를 저장합니다.
...
```

← 최적 성능 지점(Epoch 13)에서 Best Model Checkpoint 저장 완료

[모델 추론] Auto-regression 기반의 실제 텍스트 디코딩

디코딩 함수 `decode\_sequence`

```
hidden, cell = model.encoder(encoder_inputs)

# 시작 토큰 <sos> (정수 3) 주입 및 배치 차원 추가
decoder_input = torch.tensor([3]).unsqueeze(0)

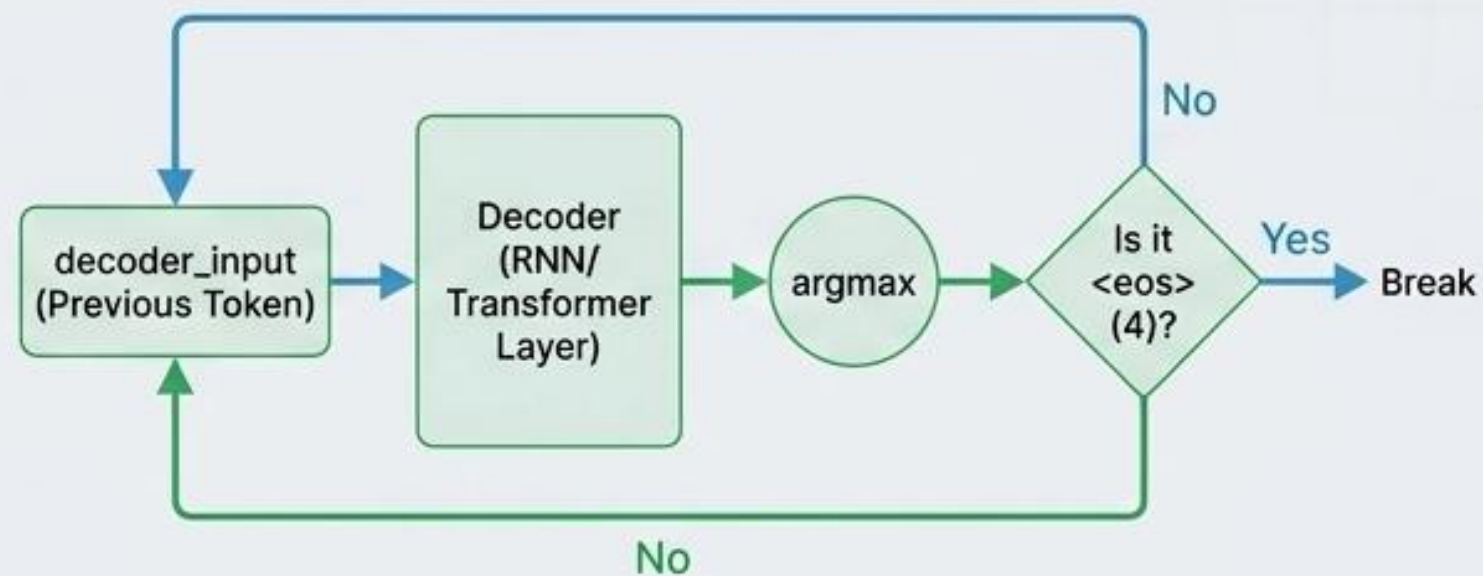
for _ in range(max_output_len):
    output, hidden, cell = model.decoder(decoder_input,
                                         hidden, cell)

    # 가장 높은 확률의 단어 인덱스 추출
    output_token = output.argmax(dim=-1).item()

    # 종료 토큰 <eos> (정수 4) 도달 시 루프 탈출
    if output_token == 4: break

    decoded_tokens.append(output_token)

    # 예측 단어를 다음 시점의 입력으로 재설정
    decoder_input = torch.tensor([output_token])
```



실제 번역 결과 확인 (Test Dataset)

입력(영어):	get your gear .
정답(프랑스어):	allez chercher votre materiel !
번역(프랑스어):	va chercher tes affaires !

완벽히 일치하지 않으나 Auto-regression을 통해 의미상 훌륭한 번역 문장을 생성해냄.

기계 번역 평가의 함정: 단순 유니그램 정밀도의 한계

평가 지표의 필요성: 언어 모델의 PPL(Perplexity)은 번역 성능을 직접적으로 대변하지 못함.
N-gram 기반의 빠르고 언어 독립적인 자동화 지표가 필요함.

[실패 사례 - Example 2] 단순 유니그램 정밀도의 치명적 오류

Candidate 문장 (모델의 번역):	the the the the the the the
Reference 1 (실제 정답 1):	the cat is on the mat
Reference 2 (실제 정답 2):	there is a cat on the mat

Step1: **계산법:** Candidate 단어 중 Reference에 존재하는 단어 수 / Candidate 총 단어 수

Step2: **판정:** Candidate의 모든 'the'가 Reference 문장에 존재함

Step3: **점수 산출:** $7 / 7 = 1.0$ (최고점 부여)



핵심 과제: 중복 단어로 인한 점수 부풀리기를 방지하고, 단어의 순서를 평가에 반영해야 함 → BLEU 스코어의 등장

[BLEU 평가] 중복 카운트 클리핑을 통한 보정된 정밀도(Modified Precision)

수학적 보정 원리 (Clipping)

단어가 Reference에서 최대 몇 번 등장했는지(Max_Ref_Count) 산출

Candidate의 단어 카운트가 Max_Ref_Count를 초과하면, 최대치로 클리핑(제한)

Before: "the 카운트 = 7" → After: "the 카운트 보정 = 2 (Reference의 최대 등장 횟수)"

Result: 최종 점수 = $2 / 7 \approx 0.285$ 로 하락 (정상화 완료)

Python Implementation

```
def simple_count(tokens, n):  
    # 단순 n-gram 개수 산출  
    return Counter(ngrams(tokens, n))
```

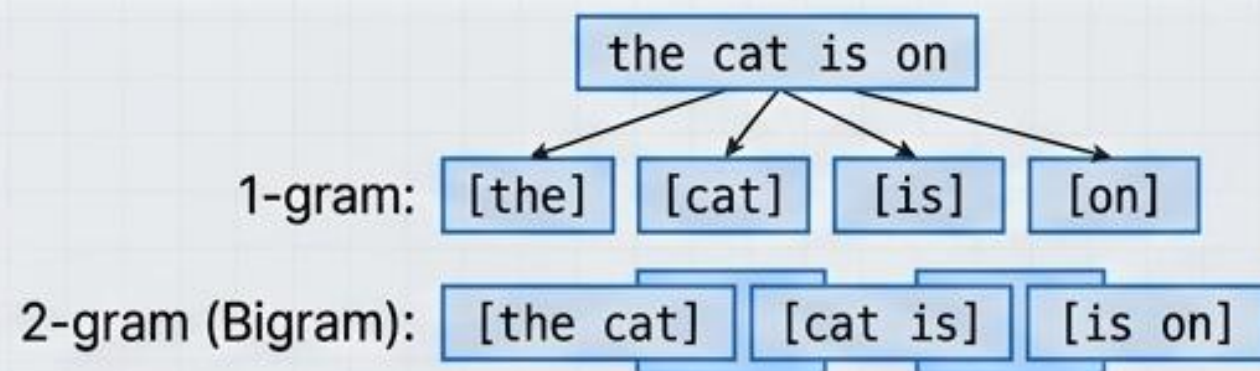
```
def count_clip(candidate, reference_list, n):  
    # 각 Reference를 순회하며 max_ref_cnt_dict 계산  
    # min(ca_cnt, max_ref_cnt_dict) 적용하여 클리핑  
    return {n_gram: min(...) for n_gram in ca_cnt}
```

```
def modified_precision(candidate, reference_list, n):  
    clip_cnt = count_clip(...) max_ref_cnt_dict  
    total_cnt = simple_count(...).total_cnt  
  
    # 클리핑 카운트 총합 / Candidate 전체 카운트 총합  
    return sum(clip_cnt) / sum(total_cnt)
```


[BLEU 평가] N-gram 확장과 짧은 문장에 대한 브레버티 패널티(Brevity Penalty)

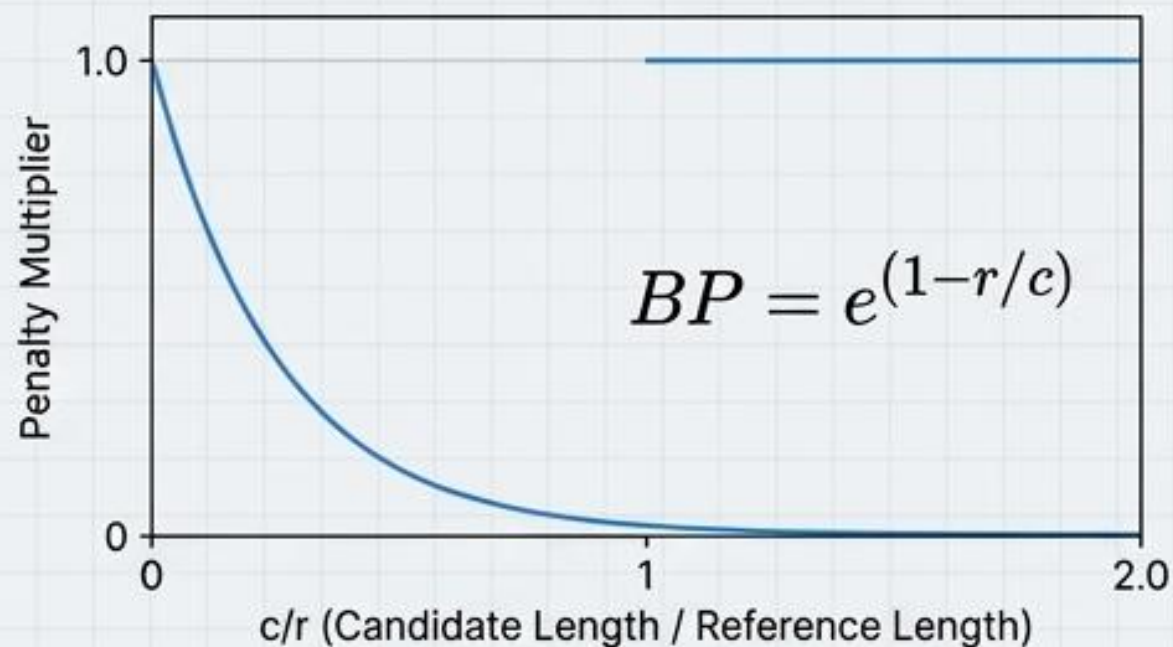
1. 순서 고려: N-gram 정밀도 확장

- 단어의 순서(문법)를 평가하기 위해 유니그램(1-gram)을 넘어 Bigram, Trigram, 4-gram까지 확장 결합.



2. 길이 보정: 브레버티 패널티 (BP)

- 비정상적으로 짧은 문장이 높은 정밀도를 받는 것을 방지 (예: Candidate 번역이 단순히 'it is' 인 경우 정밀도 1.0 오류 방지)



```
def brevity_penalty(candidate, reference_list):
    ca_len = len(candidate)
    ref_len = closest_ref_length(candidate, reference_list)

    if ca_len > ref_len:
        return 1 # 패널티 없음
    elif ca_len == 0:
        return 0
    else:
        return np.exp(1 - ref_len/ca_len) # 기하급수적 감점
```


[BLEU 평가] 최종 스코어 산출 및 NLTK 패키지 검증

$$BLEU = BP \times \exp \left(\sum_{n=1}^4 w_n \log p_n \right)$$

각 n-gram 정밀도에 균등 가중치(weights=[0.25, 0.25, 0.25, 0.25]) 적용 및 로그 합산 지수 변환

```
def bleu_score(candidate, reference_list,
               weights=[0.25, 0.25, 0.25, 0.25]):
    bp = brevity_penalty(candidate, reference_list)
    p_n = [modified_precision(...) for n in range(1, 5)]

    score = np.sum([w * np.log(p) for w, p in
                    zip(weights, p_n) if p != 0])
    return bp * np.exp(score)
```

커스텀 구현 vs NLTK 라이브러리 교차 검증

```
> 입력 문장: It is a guide to action which ensures that
the military always obeys the commands of the party
>
> 실습 코드의 BLEU 결과:
0.5045666840058485
>
> 패키지 NLTK의 BLEU 결과 (nltk.translate.bleu_score):
0.5045666840058485
```

결론: 스크래치부터 구현한 토큰화, 패딩, Seq2Seq 아키텍처 및 자체 BLEU 평가 수식이 산업 표준 라이브러리와 완벽히 일치함을 증명 완료.